Manil Bastola
Stephanie Mertz
CSE 566S
Homework 2

All experiments in this paper were performed on Wustl Linux Lab server and a personal laptop. The former will be termed as “Lab Machine” and the later as “PC” throughout the paper. The PC specs include 64 bit-Ubuntu 16.04 OS with:

Hardware:

HDD : 0.5TB SSD
vendor_id : GenuineIntel
cpu family : 6
model : 37
model name : Intel(R) Core(TM) i5 CPU M 450 @ 2.40GHz
stepping : 5
microcode : 0x2
cpu MHz : 1199.000
cache size : 3072 KB
physical id : 0
siblings : 4
cpu cores : 2

Memory:

	total	used	free	shared	buffers	cached
Mem:	3906500	3105516	800984	92340	52168	710244
-/+ buffers/cache:		2343104	1563396			
Swap:	7999484	31268	7968216			

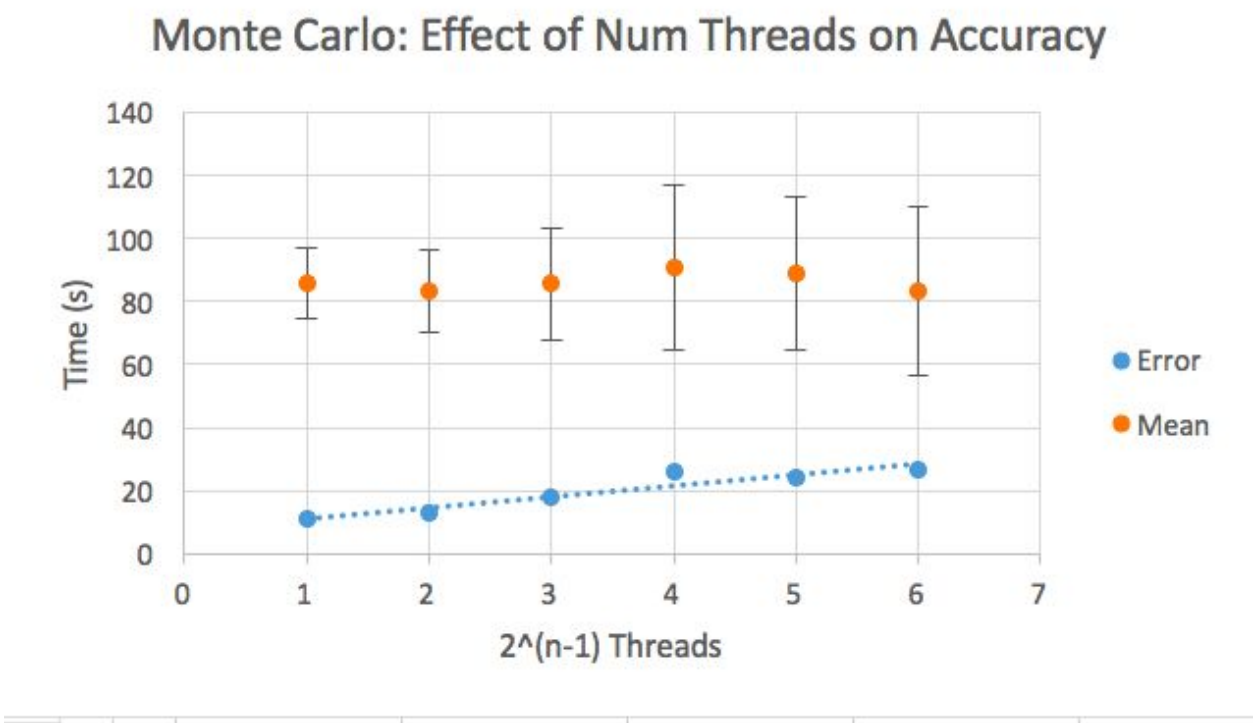
Explain why the Monte Carlo version does or does not perform as well as the given version of the code, especially when multiple threads are used. Is this what you expected?

The monte carlo algorithm is based upon “random walk” i.e. markov process whereby in each next iteration (i.e. next step in walk) is dependent on the current state only. Our implementation is based on UWHPSC implementation of Monte Carlo for Laplace equations (Bitbucket: `UWHPSC/codes/project/laplace_mc.py`). Here, we perform “many_walks” form each i,j position in the matrix *w* where each walk is a random walk till one hits the boundary. When a boundary is hit, we assign the the result of that walk to the boundary value. The final value stored in *w*[i][j] is the average value across all such many_walks. Hence the walks from each starting i,j pair is independent, which should make this algorithm easy to parallelize across different processors. Because of this, we expect OpenMP to significantly speed up the program and for performance to be greater than the other version. We do not expect the implementation in

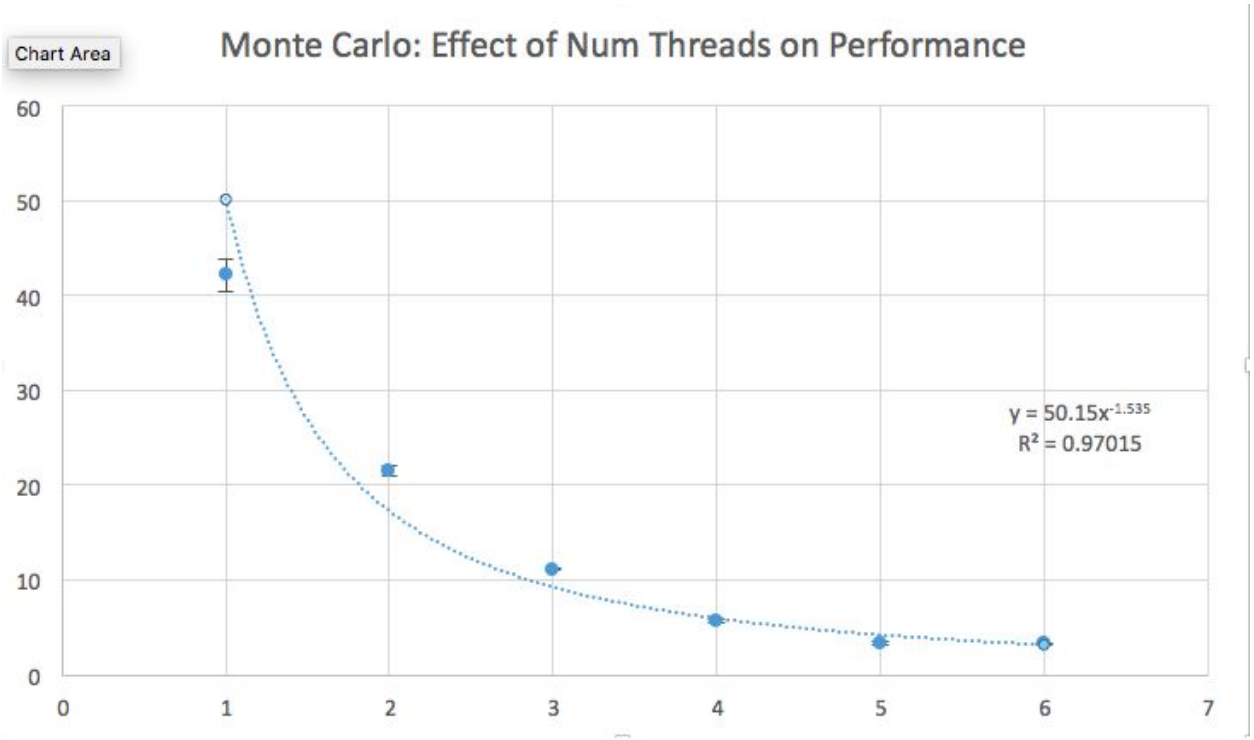
heated_plate_open_mp.c to be as parallelizable as the monte carlo approach because of dependencies along the neighbouring values while taking the averages.

However, parallelization in monte carlo also requires parallelization in the random number generator, rand_r. For the walks of each i,j position, we expect accuracy to decrease.[3] This is because the seeds used to randomly generate the numbers on each machine might line up, thereby decreasing accuracy.[3] There are other ways to randomly generate numbers in parallel that have greater accuracy such as SPRNG.[4] However, for the sake of our implementation with rand_r(), we expect accuracy to decline i.e. high variability in the data is expected.

To verify that our implementation is correct, we compare the average temperature of the plate with boundary conditions [100.0, 100.0, 100.0, 0.0]. The mean temperature value given by heated_plate_open_mp.c is 74.94 degrees while of the monte_carlo.c is 74.3 ± 3.1 degrees. Additionally, we tried another boundary condition of [100.0, 200.0, 50.0, 0.0] for high variability of temperature across the plate. This gave mean temperature on the heated_plate_open_mp.c as 87.37 degrees. The result of monte_carlo.c is 86.9 ± 3 degrees. Hence, in terms of the mean, both algorithms are comparable. Since the monte carlo approach introduces randomness and not as properly in the way discussed above, we expect for each trial to deviate on the mean more than the analytical approach would result in. To prove this, we collected data on the mean value returned by the program and displayed error bars. They appeared to grow in size, but for clarity, we graphed them as well.



Monte carlo simulations can get very slow if the step size of each walk is not considered properly. In our implementation we set it to 20 and set the max_step to 1000. These values have been fine tuned for this problem of size N=M=500 so that we get majority of of our random walks to reach boundary before max_steps is reached (in which case, the random walk is wasted since it gets no boundary value). However, monte carlo simulations do great with large num thread size since they are highly parallelizable which is shown by figure 1 above. Compare this to figure in question 5) where we do not see such linear relationships in decreasing run time with increasing number of threads as the figure below. In terms of timing, for numThread = 1, the monte carlo approach takes twice as long as expected.



1. How does Scalasca compare to the profilers we saw on the previous assignment (gprof, operf, gperftools)? What about the timing hooks included in the code? Do they all agree? Why or why not?

Scalasca is a performance analysis tool specifically for analyzing how programs run in parallel. It is made with large-scale programs in mind. It works by taking performance measurements at runtime that are then analyzed after the program has completed ("post mortem). Scalasca is particularly useful in identifying when the program is in a wait state[1]. This can help the user more efficiently allocate work to each processor to boost overall performance.

We ran heated_plate_scalasca.sh on the lab machine. The result were cube directories. Each such directory when ran with scalasca -examine flag produced nice gui interface with performance metrics. The interface was divided into three components of metric tree, call tree

and system tree. The metric tree showed gprof-like but more detailed evaluations of the program. The call tree extended the information to perf and oprofile-like program's stack traces. However the system tree showed the workings of parallel nodes such as number of master /slave threads which we didn't see in any of the tree profilers before.

In order to examine accuracy of scalasca we ran 5 sets of experiments of heated_plate_run_scalasca.sh. We set num_threads = 2 for all.We took notes of overall computation time and the number of visits. For N=M=500, the average time reported by scalasca is 12.97 ± 0.24 s. For same experiment conditions, the average time reported by grof and individual timing hook are 12.46 ± 0.11 s and 6.45 ± 0.12 s respectively.

One can observe that the result of gprof matches with the timing given by scalasca. Although the variance was higher in scalasca than gprof which is surprising. Meanwhile, both gprof and scalasca measured around 12.5 seconds while timing hooks show entirely different result. The result shown is in the range of 6.5 seconds which is about half the value shown by both profilers.

The individual timing hooks were put at the beginning and at the end of the main function. The time was obtained using omp_get_wtime ( ) function. We initially thought that since our num_thread = 2, omp_get_wtime () function might just be dividing the time by number of threads. However, this is not the case since we verified the results using sys/time.h gettimeofday() function which was around 6.5s as well.

We then thought of the difference in time due to instrumentation time (setup, sampling, etc.) of the two profilers. However this is not the case. We added a simple wrapper:

```
!/bin/bash

time1=$(date +%s%3N)
./heated_plate
time2=$(date +%s%3N)
echo $(( $time2 - $time1 ))
```

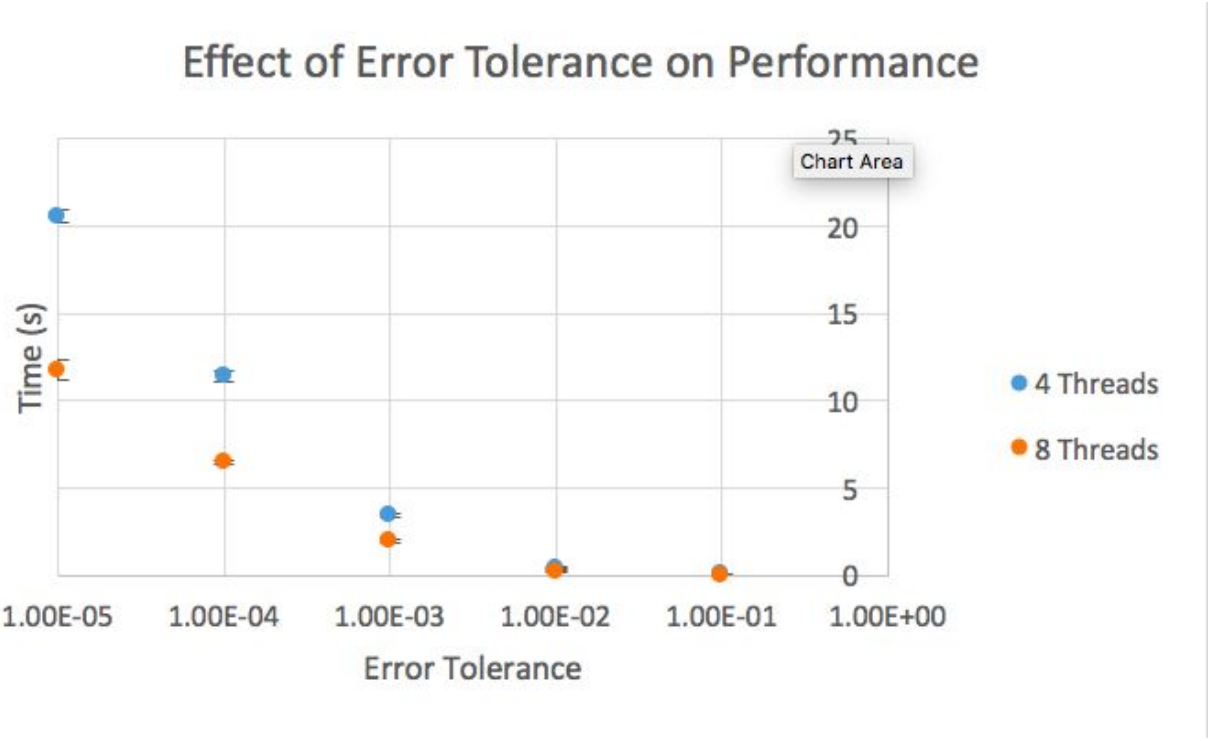
that tested total time of execution of gprof debugger enabled program heated_plate. The result of echo in the last line was around 6.5 s. Hence, the profilers somehow "think" they spent 12.5 seconds when the running time of the entire program was 6.5s.

Finally, the result of number of visits (oscillations) by scalasca was consistently 4.07e5 for all 5 trials. We think that such result has been heavily rounded off by the profiler since one consistent number is not expected.

4. *What is the performance dependence upon the size of the plate? The error tolerance?*

For this problem we chose to understand the effect of error tolerance on the *heated_plate_open_mp.c* 's run time performance. We hypothesize that increasing error tolerance would increase the time taken since there would be additional calculations needed to reach that error tolerance. We also think that this relationship is linear since the error tolerance increases the number of iterations that the algorithm undergoes so that an i.jth point in the plate is epsilon close to its neighboring point. At max, this will induce constant factor increase in time. We also hypothesise that as long as the error tolerance is low (< 0.1), the accuracy will remain relatively unchanged due to the scale of the problem. The mean temperature is in the range of 10s. Hence the scale of the error is 100 times less than the scale of the values in the plate which is averaged over M^2 number of points.

For the experiment we did 6 trials of different epsilons ranging from 0.1 to 1e-5. We picked constant num thread. The figure below shows linear relationship as expected:



5. *How does it perform as the number of threads is varied? Be sure to collect samples that use more threads than available processors.*

In this problem we compare the running time of *heated_plate_open_mp.c* with increasing number of omp threads. We performed 6 trials for each numthread and took the average. We did this on both PC and the lab machine. The results are shown below:

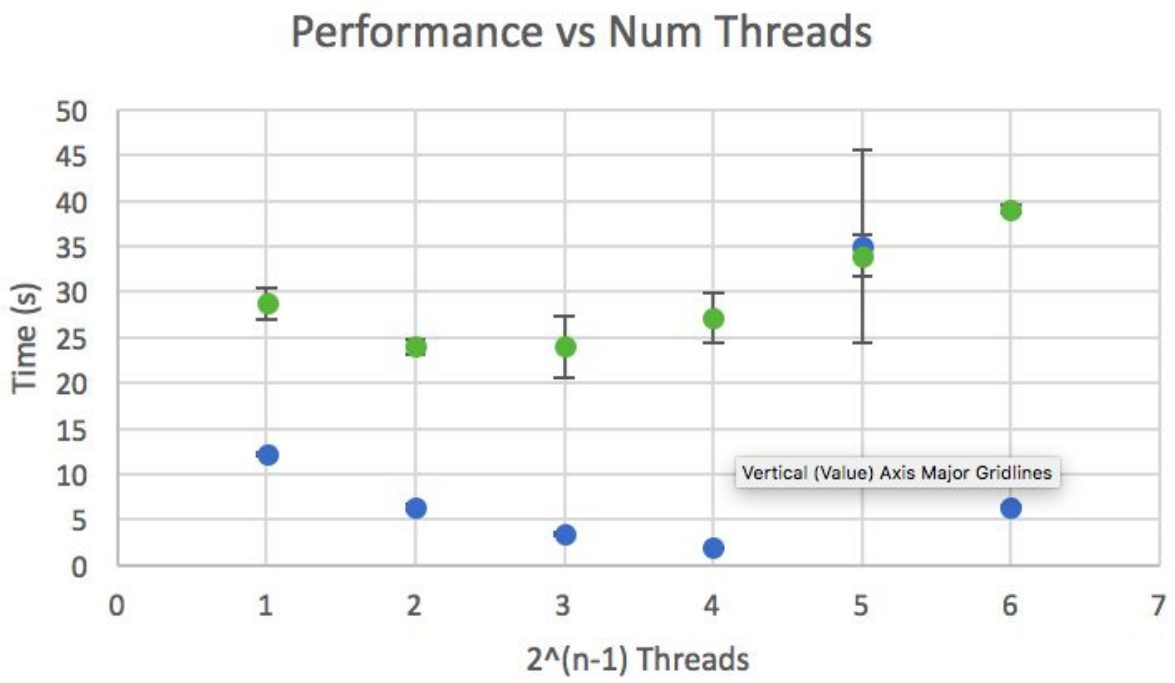


Figure: Running time vs number of threads (in 2^(n-1) scale) of heated_plate_open_mp.c on PC (green) and on lab machine (blue)

We can observe that each time we doubled the number of threads up until the number of processors we have, the time decreased by a little less than a factor of two. This is expected since the code is paralyzable, each thread can be working concurrently. However, there is a setup cost as well as communication between threads (relating to mean value). Hence the slope is in the range -1 to 0 and not perfectly -1.

As we increased the number of threads beyond the number of processors we had (16 for lab machine and 4 for PC), the performance started to decay. This is expected as threads now alternate with other threads on the same core since there are no further processors available. This causes more context switches as each thread is handled in some form of round-robin fashion by the OS. Hence, the total time begins to increase. There is an unexplainable data point at numThread = 16 (x=5) where the lab machine had really bad running time (~35s).

Bibliography

1. Geimer, Markus, Felix Wolf, Brian J. N. Wylie, Erika Ábrahám, Daniel Becker, and Bernd Mohr. "The Scalasca performance toolset architecture." Concurrency and Computation: Practice and Experience (2010): n. pag. Web. 18 Feb. 2017.
2. University of Washington, High Performance Scientific Computing. “Numerical methods for the Poisson problem”.
3. Mascagni, Michael. "Monte Carlo Methods and Partial Differential Equations: Algorithms and Implications for High-Performance Computing." Web. 18 Feb. 2017.
4. "OpenMP, number of cores and accuracy of computation with random numbers." Multithreading - OpenMP, number of cores and accuracy of computation with random numbers - Stack Overflow. N.p., n.d. Web. 18 Feb. 2017.
5. "SPRNG: Scalable Parallel Pseudo Random Number Generators Library." SPRNG: Scalable Parallel Pseudo Random Number Generators Library. N.p., n.d. Web. 18 Feb. 2017.